apache / parquet-format Public

Apache Parquet

Apache-2.0 license

1.5k stars 416 forks Activity

Star Notifications

Code Pull requests 9 Actions Projects Security Insights

master Go to file

anjakefala PARQUET-758: Add Float16/Half-float logical type (#184) ... last week 338

View code

Parquet Test passing

Parquet is a columnar storage format that supports nested data.

Parquet metadata is encoded using Apache Thrift.

The `Parquet-format` project contains all Thrift definitions that are necessary to create readers and writers for Parquet files.

Motivation

We created Parquet to make the advantages of compressed, efficient columnar data representation available to any project in the Hadoop ecosystem.

Parquet is built from the ground up with complex nested data structures in mind, and uses the record shredding and assembly algorithm described in the Dremel paper. We believe this approach is superior to simple flattening of nested name spaces.

README.md

performance impact of applying the right compression and encoding scheme to the data. Parquet allows compression schemes to be specified on a per-column level, and is future-proofed to allow adding more encodings as they are invented and implemented.

Parquet is built to be used by anyone. The Hadoop ecosystem is rich with data processing frameworks, and we are not interested in playing favorites. We believe that an efficient, well-implemented columnar storage substrate should be useful to all frameworks without the cost of extensive and difficult to set up dependencies.

Modules

The `parquet-format` project contains format specifications and Thrift definitions of metadata required to properly read Parquet files.

The `parquet-mr` project contains multiple sub-modules, which implement the core components of reading and writing a nested, column-oriented data stream, map this core onto the parquet format, and provide Hadoop Input/Output Formats, Pig loaders, and other java-based utilities for interacting with Parquet.

The `parquet-compatibility` project contains compatibility tests that can be used to verify that implementations in different languages can read and write each other's files.

Building

Java resources can be built using `mvn package`. The current stable version should always be available from Maven Central.

C++ thrift resources can be generated via make.

Thrift can be also code-generated into any other thrift-supported language.

Glossary

- Block (HDFS block): This means a block in HDFS and the meaning is unchanged for describing this file format. The file format is designed to work well on top of HDFS.
- File: A HDFS file that must include the metadata for the file. It does not need to actually contain the data.
- Row group: A logical horizontal partitioning of the data into rows. There is no physical structure that is guaranteed for a row group. A row group consists of a column chunk for each column in the dataset.
- Column chunk: A chunk of the data for a particular column. They live in a particular row group and are guaranteed to be contiguous in the file.
- Page: Column chunks are divided up into pages. A page is conceptually an indivisible unit (in terms of compression and encoding). There can be multiple page types which are interleaved in a column chunk.

Hierarchically, a file consists of one or more row groups. A row group contains exactly one column chunk per column. Column chunks contain one or more pages.

Unit of parallelization

- MapReduce - File/Row Group
- IO - Column chunk
- Encoding/Compression - Page

File format

This file and the [Thrift definition]() should be read together to understand the format.

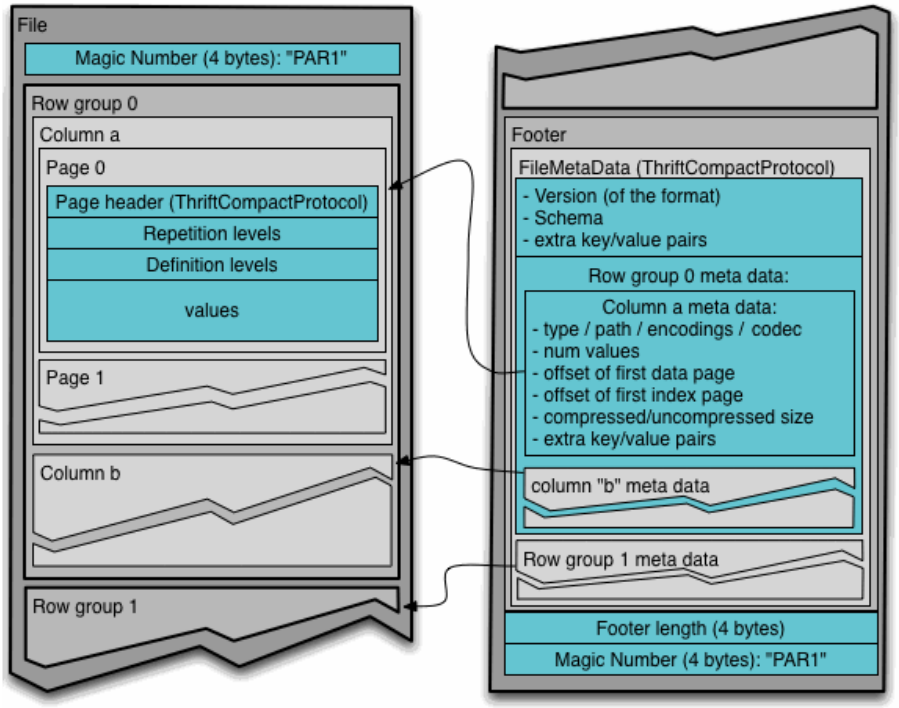
```
4-byte magic number "PAR1"
<Column 1 Chunk 1 + Column Metadata>
<Column 2 Chunk 1 + Column Metadata>
...
<Column N Chunk 1 + Column Metadata>
<Column 1 Chunk 2 + Column Metadata>
<Column 2 Chunk 2 + Column Metadata>
...
<Column N Chunk 2 + Column Metadata>
```

```
...
<Column 1 Chunk M + Column Metadata>
<Column 2 Chunk M + Column Metadata>
...
<Column N Chunk M + Column Metadata>
File Metadata
4-byte length in bytes of file metadata (little endian)
4-byte magic number "PAR1"
```

In the above example, there are N columns in this table, split into M row groups. The file metadata contains the locations of all the column metadata start locations. More details on what is contained in the metadata can be found in the Thrift definition.

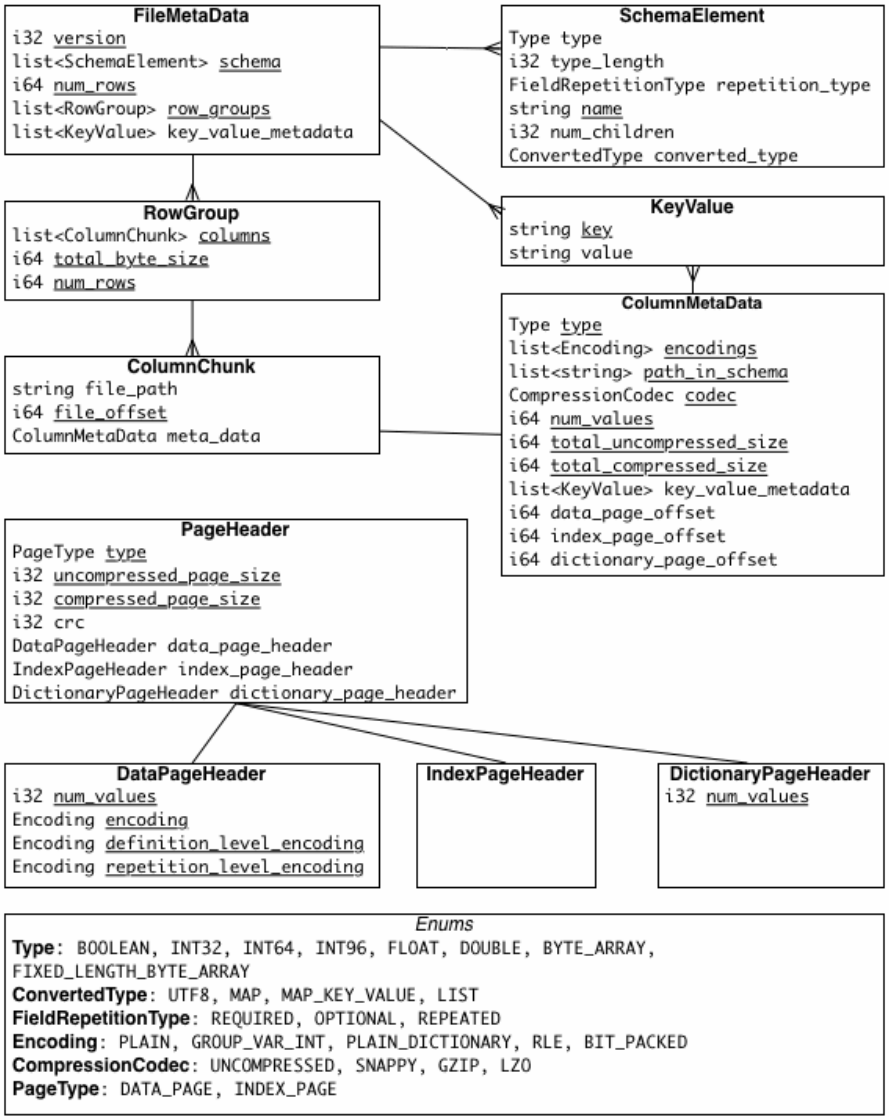
Metadata is written after the data to allow for single pass writing.

Readers are expected to first read the file metadata to find all the column chunks they are interested in. The columns chunks should then be read sequentially.



Metadata

There are three types of metadata: file metadata, column (chunk) metadata and page header metadata. All thrift structures are serialized using the TCompactProtocol.



Types

The types supported by the file format are intended to be as minimal as possible, with a focus on how the types effect on disk storage. For example, 16-bit ints are not explicitly supported in the storage format since they are covered by 32-bit ints with an efficient encoding. This reduces the complexity of implementing readers and writers for the format. The types are:

- BOOLEAN: 1 bit boolean
- INT32: 32 bit signed ints
- INT64: 64 bit signed ints
- INT96: 96 bit signed ints
- FLOAT: IEEE 32-bit floating point values
- DOUBLE: IEEE 64-bit floating point values
- BYTE_ARRAY: arbitrarily long byte arrays
- FIXED_LEN_BYTE_ARRAY: fixed length byte arrays

Logical Types

Logical types are used to extend the types that parquet can be used to store, by specifying how the primitive types should be interpreted. This keeps the set of primitive types to a minimum and reuses parquet's efficient encodings. For example, strings are stored as byte arrays (binary) with a UTF8 annotation. These annotations define how to further decode and interpret the data. Annotations are stored as `LogicalType` fields in the file metadata and are documented in LogicalTypes.md.

Sort Order

Parquet stores min/max statistics at several levels (such as Column Chunk, Column Index and Data Page). Comparison for values of a type obey the following rules:

1. Each logical type has a specified comparison order. If a column is annotated with an unknown logical type, statistics may not be used for pruning data. The sort order for logical types is documented in the LogicalTypes.md page.
2. For primitive types, the following rules apply:
   - BOOLEAN - false, true
   - INT32, INT64 - Signed comparison.
   - FLOAT, DOUBLE - Signed comparison with special handling of NaNs and signed zeros. The details are documented in the Thrift definition in the `ColumnOrder` union. They are summarized here but the Thrift definition is considered authoritative:
     - NaNs should not be written to min or max statistics fields.
     - If the computed max value is zero (whether negative or positive), `+0.0` should be written into the max statistics field.
     - If the computed min value is zero (whether negative or positive), `-0.0` should be written into the min statistics field.

     For backwards compatibility when reading files:
     - If the min is a NaN, it should be ignored.
     - If the max is a NaN, it should be ignored.
     - If the min is +0, the row group may contain -0 values as well.
     - If the max is -0, the row group may contain +0 values as well.
     - When looking for NaN values, min and max should be ignored.
   - BYTE_ARRAY and FIXED_LEN_BYTE_ARRAY - Lexicographic unsigned byte-wise comparison.

Nested Encoding

To encode nested columns, Parquet uses the Dremel encoding with definition and repetition levels. Definition levels specify how many optional fields in the path for the column are defined. Repetition levels specify at what repeated field in the path has the value repeated. The max definition and repetition levels can be computed from the schema (i.e. how much nesting there is). This defines the maximum number of bits required to store the levels (levels are defined for all values in the column).

Two encodings for the levels are supported BIT_PACKED and RLE. Only RLE is now used as it supersedes BIT_PACKED.

Nulls

Nullity is encoded in the definition levels (which is run-length encoded). NULL values are not encoded in the data. For example, in a non-nested schema, a column with 1000 NULLs would be encoded with run-length encoding (0, 1000 times) for the definition levels and nothing else.

Data Pages

For data pages, the 3 pieces of information are encoded back to back, after the page header. No padding is allowed in the data page. In order we have:

1. repetition levels data
2. definition levels data
3. encoded values

The value of `uncompressed_page_size` specified in the header is for all the 3 pieces combined.

The encoded values for the data page is always required. The definition and repetition levels are optional, based on the schema definition. If the column is not nested (i.e. the path to the column has length 1), we do not encode the repetition levels (it would always have the value 1). For data that is required, the definition levels are skipped (if encoded, it will always have the value of the max definition level).

For example, in the case where the column is non-nested and required, the data in the page is only the encoded values.

The supported encodings are described in Encodings.md

The supported compression codecs are described in Compression.md

Column chunks

Column chunks are composed of pages written back to back. The pages share a common header and readers can skip over pages they are not interested in. The data for the page follows the header and can be compressed and/or encoded. The compression and encoding is specified in the page metadata.

A column chunk might be partly or completely dictionary encoded. It means that dictionary indexes are saved in the data pages instead of the actual values. The actual values are stored in the dictionary page. See details in Encodings.md. The dictionary page must be placed at the first position of the column chunk. At most one dictionary page can be placed in a column chunk.

Additionally, files can contain an optional column index to allow readers to skip pages more efficiently. See PageIndex.md for details and the reasoning behind adding these to the format.

Checksumming

Pages of all kinds can be individually checksummed. This allows disabling of checksums at the HDFS file level, to better support single row lookups. Checksums are calculated using the standard CRC32 algorithm - as used in e.g. GZip - on the serialized binary representation of a page (not including the page header itself).

Error recovery

If the file metadata is corrupt, the file is lost. If the column metadata is corrupt, that column chunk is lost (but column chunks for this column in other row groups are okay). If a page header is corrupt, the remaining pages in that chunk are lost. If the data within a page is corrupt, that page is lost. The file will be more resilient to corruption with smaller row groups.

Potential extension: With smaller row groups, the biggest issue is placing the file metadata at the end. If an error happens while writing the file metadata, all the data written will be unreadable. This can be fixed by writing the file metadata every Nth row group. Each file metadata would be cumulative and include all the row groups written so far. Combining this with the strategy used for rc or avro files using sync markers, a reader could recover partially written files.

Separating metadata and column data.

The format is explicitly designed to separate the metadata from the data. This allows splitting columns into multiple files, as well as having a single metadata file reference multiple parquet files.

Configurations

- Row group size: Larger row groups allow for larger column chunks which makes it possible to do larger sequential IO. Larger groups also require more buffering in the write path (or a two pass write). We recommend large row groups (512MB - 1GB). Since an entire row group might need to be read, we want it to completely fit on one HDFS block. Therefore, HDFS block sizes should also be set to be larger. An optimized read setup would be: 1GB row groups, 1GB HDFS block size, 1 HDFS block per HDFS file.
- Data page size: Data pages should be considered indivisible so smaller data pages allow for more fine grained reading (e.g. single row lookup). Larger page sizes incur less space overhead (less page headers) and potentially less parsing overhead (processing headers). Note: for sequential scans, it is not expected to read a page at a time; this is not the IO chunk. We recommend 8KB for page sizes.

Extensibility

There are many places in the format for compatible extensions:

- File Version: The file metadata contains a version.
- Encodings: Encodings are specified by enum and more can be added in the future.
- Page types: Additional page types can be added and safely skipped.

Contributing

Comment on the issue and/or contact the parquet-dev mailing list with your questions and ideas. Changes to this core format definition are proposed and discussed in depth on the mailing list. You may also be interested in contributing to the Parquet-MR subproject, which contains all the Java-side implementation and APIs. See the "How To Contribute" section of the Parquet-MR project

Code of Conduct

We hold ourselves and the Parquet developer community to a code of conduct as described by Twitter OSS: https://github.com/twitter/code-of-conduct/blob/master/code-of-conduct.md.

License

Copyright 2013 Twitter, Cloudera and other contributors.

Licensed under the Apache License, Version 2.0: http://www.apache.org/licenses/LICENSE-2.0

Releases

19 tags

Packages

No packages published

Contributors 63

+ 52 contributors

Languages

- **Java** 41.5%
- **Thrift** 35.5%
- **Python** 14.7%
- **Shell** 7.2%
- **Makefile** 1.1%